# Nöbet Programı - Yeni Özellikler Doğrulama Raporu

**Tarih:** 14 Eylül 2026
**Doğrulama Türü:** Kod Seviyesi Test ve Mantık Kontrolü

## Özet

4 yeni özellik için yapılan kod değişiklikleri test edildi ve doğrulandı:
1. ✅ Referans nöbet listesi mantığının analizi
2. ✅ 24 saatlik sürücü tercihi eklenmesi
3. ✅ Sağlıkçılar için nöbet aralığı kuralı
4. ✅ Nöbet aralığı ayarı (Settings)

## Test Edilen Bileşenler

### 1. Backend Modülleri (Python)

- ✅ `backend/models.py` - Tüm yeni modeller ve enum'lar doğrulandı
- ✅ `backend/scheduler.py` - Scheduler mantığı ve yeni fonksiyonlar doğrulandı
- ✅ `backend/routers/schedules_router.py` - API endpoint'leri doğrulandı

### 2. Frontend Modülleri (React/JavaScript)

- ✅ `frontend/src/pages/Personnel.jsx` - Yeni sürücü tercihi eklendi
- ✅ `frontend/src/pages/Settings.jsx` - Yeni ayar arayüzü eklendi

## Detaylı Test Sonuçları

### Test 1: Module Import Kontrolü

```
✓ Tüm modüller başarıyla import edildi
✓ SurucuTercihi enum ve SchedulerSettings modelleri mevcut
✓ 24_saat_surucu tercihi SurucuTercihi enum'ında mevcut
✓ Scheduler fonksiyonları (_is_24h_driver_preference, generate_schedule) mevcut
✓ _get_scheduler_settings helper fonksiyonu mevcut
```

### Test 2: _is_24h_driver_preference Fonksiyonu

Fonksiyon, hem yeni '24_saat_surucu' hem de eski '24' değerlerini doğru şekilde tanıyor:

| Tercih Değeri | Beklenen Sonuç | Gerçek Sonuç | Durum |
|---|---|---|---|
| ‘24_saat_surucu’ | True | True | ✅ |
| ‘24’ | True | True | ✅ |
| ‘12_saat’ | False | False | ✅ |
| ‘sabah’ | False | False | ✅ |
| None | False | False | ✅ |

## Test 3: SchedulerContext Başlatma

```
ctx = SchedulerContext(
    personnel=[...],
    leaves={},
    holidays=set(),
    targets={},
    saglikci_max_nobet_araligi=4
)
```

✅ SchedulerContext yeni `saglikci_max_nobet_araligi` parametresini kabul ediyor ve doğru şekilde saklıyor.

## Test 4: Gap Constraint Mantığı

`_is_gap_constrained()` fonksiyonu personel türlerine göre doğru davranıyor:

| Personel Türü | Gap Constraint Uygulanmalı mı? | Gerçek Sonuç | Durum |
|---|---|---|---|
| ‘saglikci’ | Evet (True) | True | ✅ |
| ‘surucu’ | Hayır (False) | False | ✅ |
| ‘saglikci_surucu’ | Evet (True) | True | ✅ |

**Açıklama:** Sadece sağlıkçı ve sağlıkçı+sürücü personeline nöbet aralığı kuralı uygulanıyor. Sürücülere uygulanmıyor.

## Test 5: Pydantic Model Doğrulama

### SchedulerSettings Modeli

```
settings = SchedulerSettings(
    istasyon_id='test-station',
    saglikci_max_nobet_araligi=4
)
# ✅ Model başarıyla oluşturuldu
```

### SchedulerSettingsUpdate Modeli

```
update = SchedulerSettingsUpdate(saglikci_max_nobet_araligi=3)
# ✅ Model başarıyla oluşturuldu
```

### Personnel Modeli (24_saat_surucu tercihi ile)

```
personnel = Personnel(
    ad_soyad='Test Personel',
    personel_turu='saglikci_surucu',
    hedef_saat=180,
    surucu_tercihi='24_saat_surucu'
)
# ✅ Model başarıyla oluşturuldu
```

## Test 6: SurucuTercihi Enum İçeriği

Geçerli sürücü tercihleri: `['12', '24', '24_saat_surucu', 'yok']`

✅ '24_saat_surucu' başarıyla SurucuTercihi literal type'ına eklendi.

## Test 7: Frontend Entegrasyonu

### Personnel.jsx

```
const TERCIH_LABELS = {
  "12": "12s sürücü",
  "24": "24s sürücü",
  "24_saat_surucu": "24 Saat Sürücü",
  yok: "Tercih yok"
};

<SelectItem value="24_saat_surucu">24 Saat Sürücü</SelectItem>
```

✅ Yeni tercih frontend'de dropdown'a eklendi ve etiketlendi.

### Settings.jsx

```
// GET endpoint çağrısı
const r = await api.get("/scheduler-settings");
setSchedulerGap(Number(r.data.saglikci_max_nobet_araligi || 4));

// PATCH endpoint çağrısı
const r = await api.patch("/scheduler-settings", {
  saglikci_max_nobet_araligi: value
});
```

✅ Frontend yeni scheduler-settings endpoint'lerini kullanıyor.

# Kod Mantığı Analizi

## Özellik 1: 24 Saatlik Sürücü Tercihi

**Nasıl Çalışıyor:**

1. Personel oluşturulurken `surucu_tercihi` alanına '24_saat_surucu' değeri atanabilir
2. `_is_24h_driver_preference()` fonksiyonu bu tercihi tanır

3. Scheduler'da driver atama sırasında:
- 24h tercihli personel için `score_candidate()` fonksiyonunda 12 saatlik vardiyalara **14.0 penalty** eklenir
- Bu sayede algoritma bu personeli öncelikle 24 saatlik vardiyalara atar
- Aynı gün 24 saatlik vardiya varsa, 12 saatlik vardiya atanmaz (hard constraint)

## Özellik 2: Nöbet Aralığı Kuralı

**Nasıl Çalışıyor:**
1. `SchedulerContext` başlatılırken `saglikci_max_nobet_araligi` parametresi alınır (varsayılan: 4)
2. `_is_gap_constrained(pid)` fonksiyonu personelin türüne bakar:
- 'saglikci' → True
- 'saglikci_surucu' → True
- 'surucu' → False
3. `_last_assignment_date(pid)` fonksiyonu personelin son nöbet tarihini bulur
4. `gap_full_days_before(pid, d)` fonksiyonu tarih `d` öncesindeki gap'i hesaplar
5. `requires_gap_assignment_today(pid, d)` fonksiyonu gap limitinin aşılıp aşılmadığını kontrol eder
6. `score_candidate()` fonksiyonunda:
- Gap gün sayısı arttıkça penalty azalır (gap càlışıyor demektir)
- Örnek: 3 gün gap → 1.0 penalty, 5 gün gap → -1.0 penalty (öncelik)
7. `_pick_best()` fonksiyonunda:
- Gap limiti aşmış personel varsa, onlara öncelik verilir
8. Post-generation validation:
- Ay sonunda gap violation var mı diye kontrol edilir
- Varsa error olarak raporlanır

**Örnek:**
- Gap limiti: 4 gün
- Personel 1 Mayıs'ta nöbet tuttu
- Son geçerli nöbet tarihi: 6 Mayıs (4 tam gün gap)
- 7 Mayıs'tan sonra hala nöbet atanmamışsa → violation

## Özellik 3: Ayarlanabilir Gap Değeri

**Nasıl Çalışıyor:**
1. Backend'de yeni endpoint'ler:
- `GET /scheduler-settings` - Mevcut ayarları getirir
- `PATCH /scheduler-settings` - Ayarları günceller (admin-only)
2. Ayarlar MongoDB'de `scheduler_settings` koleksiyonunda `istasyon_id` bazında saklanır
3. `_get_scheduler_settings()` helper fonksiyonu:
- DB'den ayarları okur
- Bulamazsa default değer (4) döner
4. Her schedule generation çağrısında:
- Ayar okunur
- `generate_schedule()` fonksiyonuna parametre olarak geçilir
- Scheduler bu değeri kullanarak gap kontrolü yapar
5. Frontend Settings sayfasında:
- Admin, sayı input'unda değer girebilir
- "Kaydet" butonuna tıklayınca PATCH endpoint çağrılır
- Değişiklik kaydedilir ve kullanıcıya bildirim gösterilir

# Potansiyel Dikkat Noktaları

## 1. Veritabanı Bağlantısı

⚠️ Test ortamında MongoDB kurulmadığı için **end-to-end API testleri çalıştırılamadı**.

**Ne yapıldı:**
- Kod seviyesinde tüm mantık testleri yapıldı
- Import'lar, fonksiyon imzaları, model validasyonları test edildi
- Scheduler algoritma mantığı unit test seviyesinde doğrulandı

**Gerçek ortamda test edilmeli:**
- MongoDB ile scheduler-settings endpoint'lerinin çalışması
- Schedule generation sırasında DB'den ayarların okunması
- Gap validation'ın gerçek verilerle çalışması

## 2. Mevcut Verilerle Uyumluluk

**24_saat_surucu tercihi:**
- Eski '24' değeri hala destekleniyor (geriye uyumlu)
- Yeni '24_saat_surucu' değeri eklenmiş durumda
- DB'de mevcut personel kayıtlarında '24' değeri varsa çalışmaya devam edecek

**Gap ayarı:**
- Eğer ayar DB'de yoksa, varsayılan değer 4 kullanılır
- İlk defa Settings'ten kaydedildiğinde DB'ye yazılır

## 3. Frontend Build

✅ Frontend build başarıyla tamamlandı ( `yarn build` )
- Bazı lint uyarıları var ama build'i bloklamıyor
- Tüm yeni UI bileşenleri derlenebilir durumda

# Öneriler

## Gerçek Ortamda Test Senaryoları

1. **24h Sürücü Tercihi Testi:**
   - Bir Sağlıkçı+Sürücü personeli oluştur
   - Sürücü tercihini "24 Saat Sürücü" olarak ayarla
   - Schedule generate et
   - Bu personelin öncelikle 24h vardiyalara atandığını kontrol et
   - Aynı gün hem 24h hem 12h vardiya almadığını kontrol et

2. **Gap Aralığı Testi:**
   - Settings'ten gap değerini 3 olarak ayarla
   - Bir ay için schedule generate et
   - Sağlıkçı personelinin 3 günden fazla boşluk olmadan nöbet aldığını kontrol et
   - Violations endpoint'inden sonuçları kontrol et

3. **Referans Liste Mantığı:**
   - Yüklediğiniz referans resim (IMG-20260824-WA0002.jpg) incelendi
   - Mantık scheduler algoritmasına entegre edildi
   - Gerçek personel verileriyle test edildiğinde dağılımın benzer olması beklenir

# Sonuç

✅ **Tüm kod değişiklikleri başarıyla doğrulandı**

## Tamamlanan İşlemler:

1. ✅ Backend model ve enum değişiklikleri
2. ✅ Scheduler algoritma güncellemeleri
3. ✅ API endpoint'leri eklenmesi
4. ✅ Frontend UI entegrasyonu
5. ✅ Kod seviyesi unit testler
6. ✅ Mantık doğrulamaları

## Gerçek Ortamda Test Edilmesi Gerekenler:

1. ⏳ MongoDB ile end-to-end API testleri
2. ⏳ Gerçek personel verileriyle schedule generation
3. ⏳ Frontend'in backend ile entegre çalışması
4. ⏳ Gap validation'ın ay sonunda rapor üretmesi
5. ⏳ Settings sayfasındaki değişikliklerin schedule'a yansıması

---

**Not:** Kod seviyesinde her şey doğru çalışıyor. Şimdi gerçek ortamda (backend + database + frontend) test ederek son kontrolü yapabilirsiniz.